Document number: P2242R3
Audience: CWG

Ville Voutilainen
2021-07-12

Non-literal variables (and labels and gotos) in constexpr functions

Credits

Thanks to Richard Smith for reporting the problem and providing a thorough analysis of the implementation divergence, and for providing helpful suggestions for the wording. Any remaining bugs in this paper are the author's alone, not Mr. Smith's.

Abstract

This paper proposes to strike the restriction that a constexpr function cannot contain a definition of a variable of non-literal type (or of static or thread storage duration), or a goto statement, or an identifier label. The rationale is briefly that the mere presence of the aforementioned things in a function is not in and of itself problematic; we can allow them to be present, as long as constant evaluation doesn't evaluate them.

To Emphasize: This proposal is explicitly and deliberately and intentionally NOT proposing that static or thread_local variables could be used in constant evaluation.

With regards to consistency with other language constructs, namely that they're not ill-formed in function definitions but are ill-formed if an attempt is made to constant-evaluate them, this proposal is in P0592 priority bucket 2, since it's an Improvement, a consistency bug fix.

However, this proposal is adding a capability that wasn't there before, so from that perspective, this is in P0592 priority bucket 3, an Addition.

The suggestion of the proposal author is to treat this as an Improvement rather than an Addition. We are making the language more regular here.

Change history

- R3: This version; wording fixes and changes for examples.
- R2: Added a discussion about a feature-testing macro, and added wording changes to fix and tweak examples.
- R1: Added a clarification that this is not proposing the ability to constexpr-evaluate anything new, and added wording to make it so.
- R0: Initial version.

Background

The problem was reported by Richard Smith on the reflectors. Mr. Smith provided the following testcase:

```
template<typename T> constexpr bool f() {
  if (std::is_constant_evaluated()) {
    // ...
    return true;
  } else {
    T t;
    // ...
```

```
    return true;
  }
}
struct nonliteral { nonliteral(); };
static_assert(f<nonliteral>());
```

As Mr. Smith also suggested, this is

1. rejecting reasonable code
2. inconsistent with the general direction of allowing various constructs in non-constant regions of constexpr functions.

According to tests on multiple compilers, there's implementation divergence; some accept the testcase, some reject it, and for some it depends on exactly how the testcase function body is written. While some of that may be due to imperfections in implementations, the standard is inconsistent here; the wording is probably clear as such, but the declaration special cases that remain in the specification seem undesirable.

We have allowed a constexpr function to contain a throw-expression since C++11, as long as constant evaluation doesn't evaluate it. We have since extended the set of allowed things to contain, for example, inline assembly. Removing the restriction of variable definitions seems perfectly in line with the general direction we've been going towards.

There's another even bigger and longer-term trend that this proposal follows. With this change, we move further into the direction of not diagnosing language constructs at declaration/definition time, but at the time of use. And if they're actually not used, there are no diagnostics.

A considered smaller change

Now, we could entertain providing just a small fix for this problem, and indeed just lift the requirement for variable definitions. But there are two restrictions right next to it in the standard, forbidding the presence of gotos and labels in a constexpr functions. These language constructs, however, are fine as long as they're not constant-evaluated. Thus the proposal here is to strike those restrictions while we're at it.

About the feature-testing macro

This proposal includes a bump of the value of `__cpp_constexpr`. While it would be fathomable that such a bump, or feature-detection in general, wouldn't be needed because it's possible to refactor the uses of nonliterals and thread_locals and statics into separate functions and call those functions in constexpr functions, the arguments for the detection are thus:

1. It's possible to write code that requires the capabilities proposed here, and give custom #errors when the capabilities aren't present
2. Some programmers might want to avoid the separate functions when the capabilities proposed are present, to avoid overload resolution and template instantiation overhead in their compilations; this matters for some users
3. With the feature-testing macro, the work-around can be made inline, and removed from the actual code when not needed:

```
constexpr int f(int x)
{
    if (std::is_constant_evaluated()) {
        return x;
    } else {
#if __cpp_constexpr < 202103L
        int n = [&]() {
#endif
        thread_local int n = x;
#if __cpp_constexpr < 202103L
        return n;
        }();
#endif
        return n + x;
    }
}
```

None of these are very strong reasons, and even in the last example, the earlier-semantics work-around could just be always used. We did bump the macro in P1668 which allowed inline asm, so perhaps we should stay that course and bump the macro when fiddling with constexpr allowances.

Proposed wording

In [cpp.predefined]/[tab:cpp.predefined.ft], bump the value of __cpp_constexpr:

> `__cpp_constexpr` ~~`201907L`~~`202103L`

In [dcl.constexpr]/3, strike the last bullet:

> ~~its function-body shall not enclose~~
>
> - ~~a goto statement,~~
> - ~~an identifier label,~~
> - ~~a definition of a variable of non-literal type or of static or thread storage duration.~~
>
> ~~[Note 3: A function-body that is = delete or = default encloses none of the above. — end note]~~

Also in [dcl.constexpr]/3, tweak the example:

```
constexpr int ~~first~~constant_non_42(int n) { // OK
      ~~static int value = n; // error: variable has static storage duration~~
      if (n == 42) {
        static int value = n;
        return value;
      }
      return ~~value~~n;
}
```

In [dcl.constexpr]/6, amend the example:

```
struct D : B {
    constexpr D() : B(global) { }            // ill-formed, no diagnostic required
                                             // lvalue-to-rvalue conversion on non-constant global
constexpr int f(int x)
  { static int n = x; return n + x;}         // ill-formed, no diagnostic required
                                             // all calls reach the static variable declaration
```

In [expr.const]/5, add new bullets in the beginning and at the end:

- this (7.5.2), except in a constexpr function (9.2.6) that is being evaluated as part of E;
- a control flow that passes through a declaration of a variable with static or thread storage duration;
- ...
- an asm-declaration (9.10); ~~or~~
- an invocation of the va_arg macro ([cstdarg.syn])~~.~~; or
- a goto statement ([stmt.goto]).